UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SOFTWARE DESIGN AND ARCHITECTURE (SDA)

SESSION 2

Innovating Solutions

UTMSPACE ESTD 1993 (931427-K)

01

INTRODUCTION

Intro to UML.

02

BEHAVIORAL MODEL

Use Case Diagram, Sequence Diagram, Activity Diagram and etc..

03

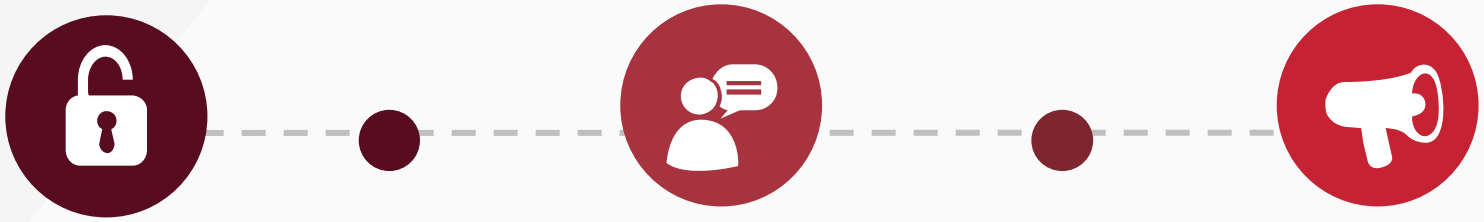
STRUCTURE MODEL

Component Diagram, Class Diagram

Unified Modeling Language

Why?

To help to communicate efficiently.



SECURITY PROOF

Always get documented then verify and validate with stakeholders.

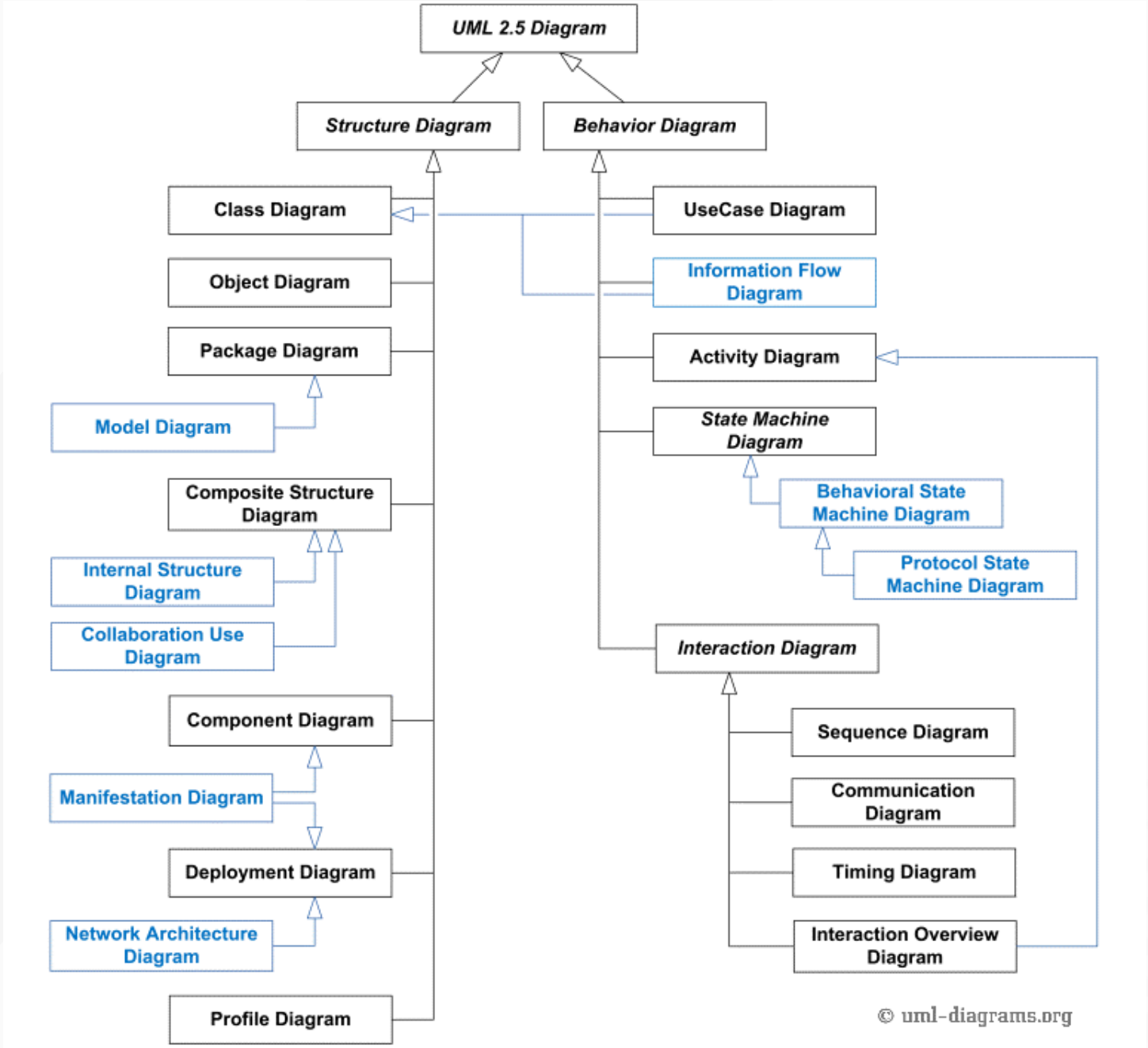
LANGUAGE

Different language proficiency.

COMPLETENESS

Clear and concise process to address all requirements.

Unified Modeling Language



References

https://www.uml-diagrams.org/

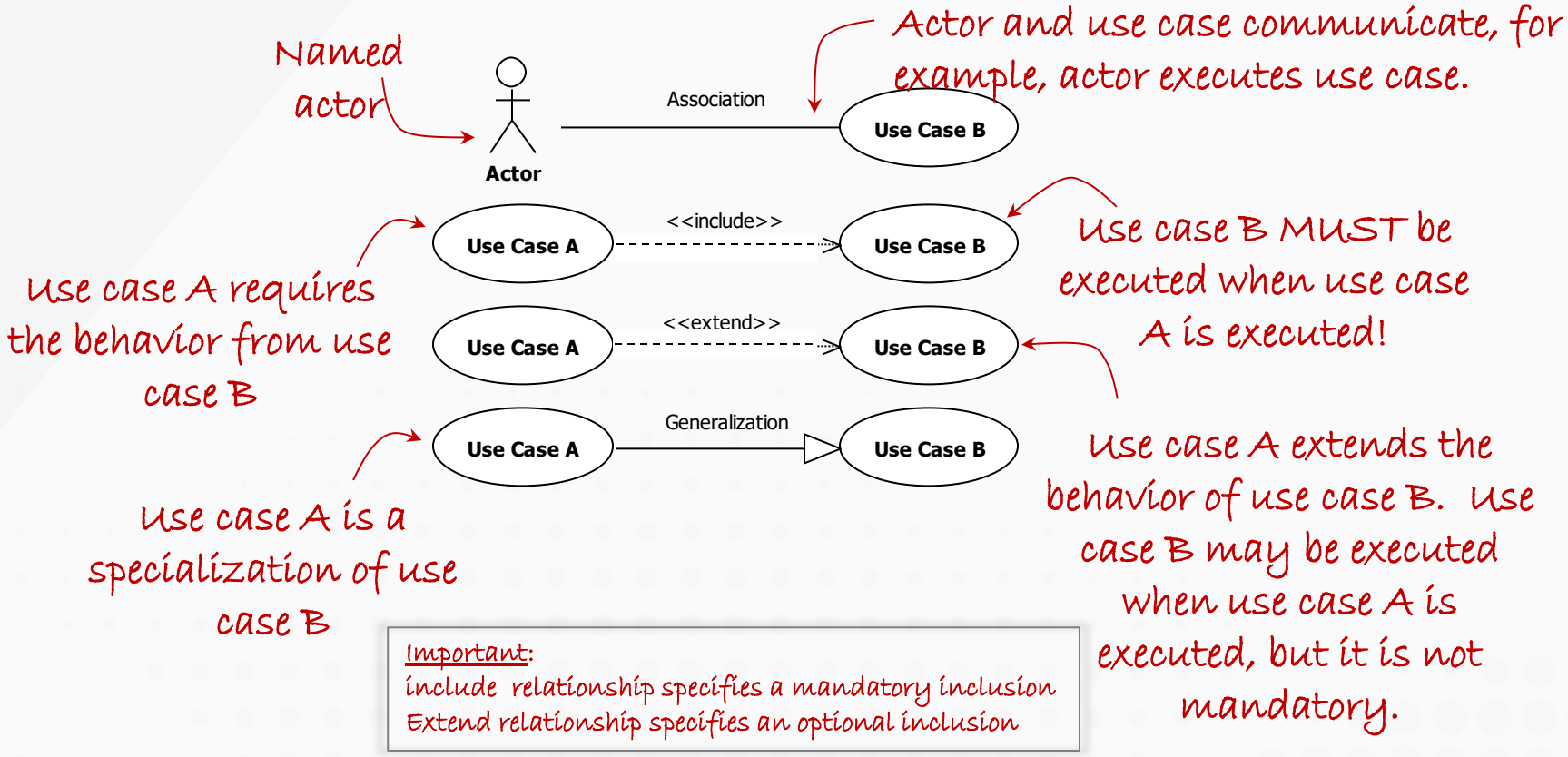
BEHAVIORAL MODEL (USE CASE DIAGRAM)

- Use case diagram
  - ✓ Behavioral diagram used to capture, specify, and visualize required system behavior.
    - Required system behavior are just requirements!
  - ✓ The main elements of use case diagrams are *actors*, *use cases*, and the *relationships* connecting them together.

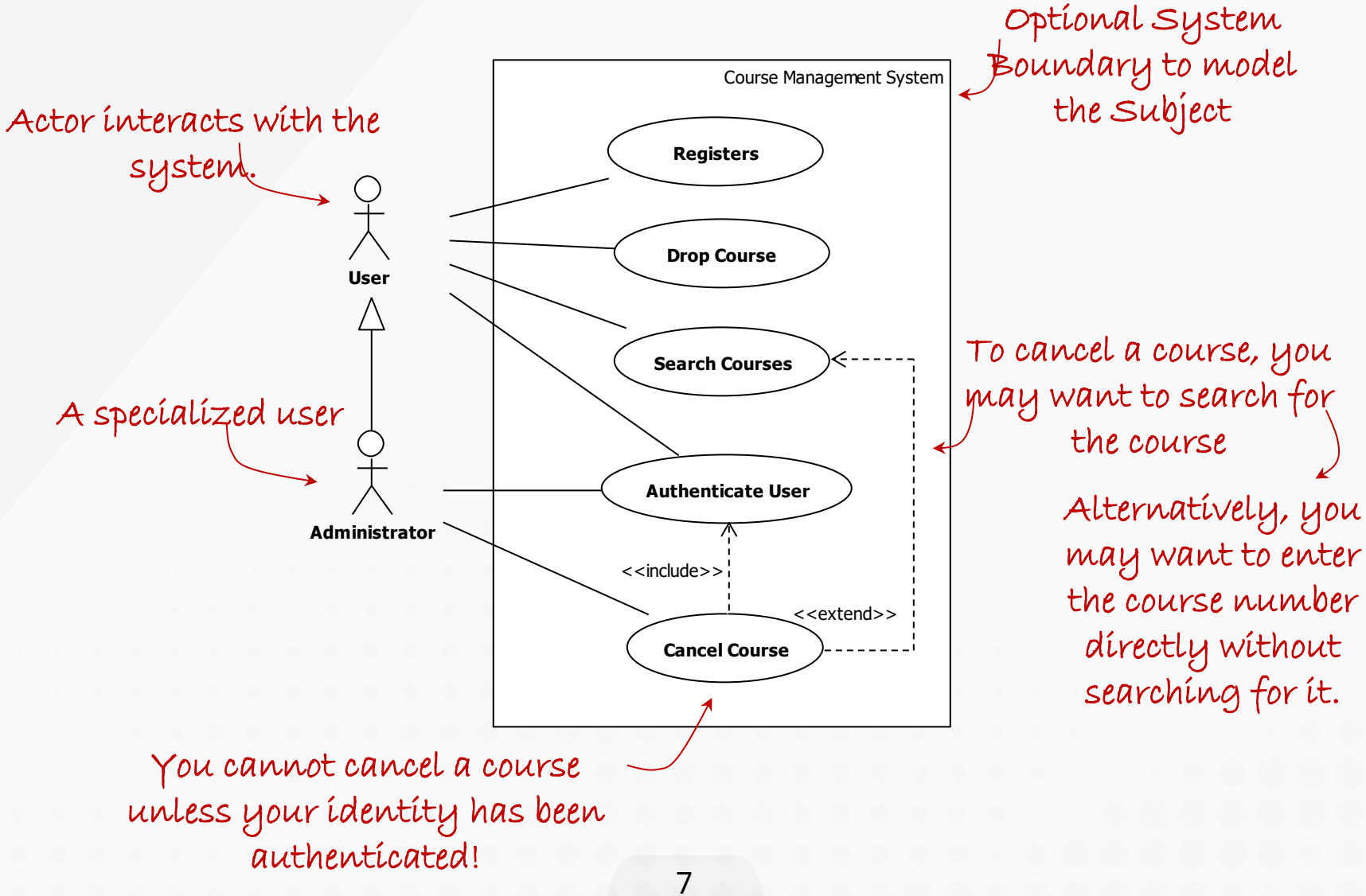
- Use cases are entities used in use case diagrams to specify the required behavior of a system.
  - ✓ They provide the means to capture, model, and visualize the systems' required behavior.
  - ✓ They do this without any knowledge of programming technology, so that different stakeholders with different backgrounds can reason about the system. Common UML relationships applied in use case diagrams

BEHAVIORAL MODEL (USE CASE DIAGRAM)

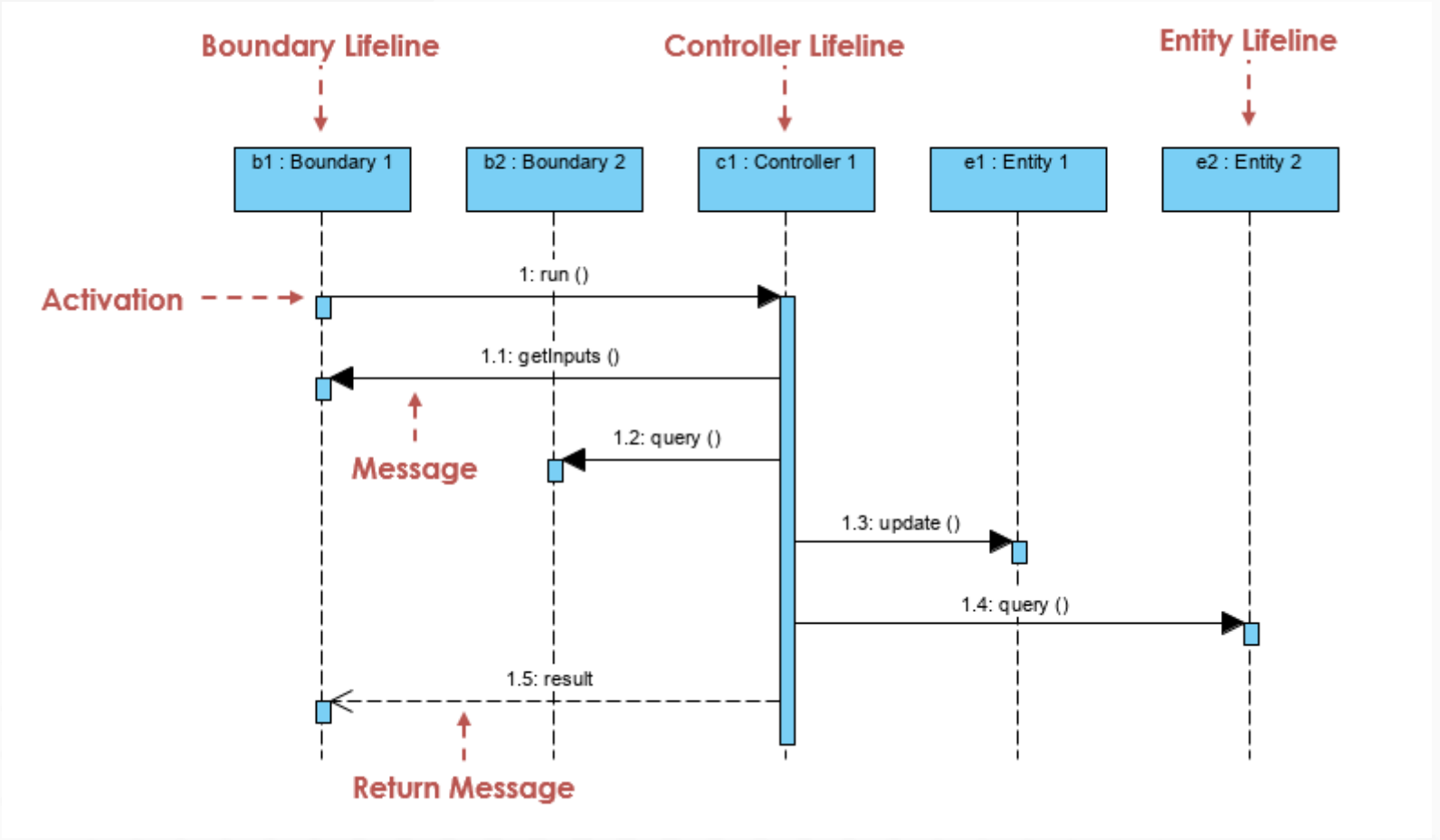
- Common UML relationships applied in use case diagrams.



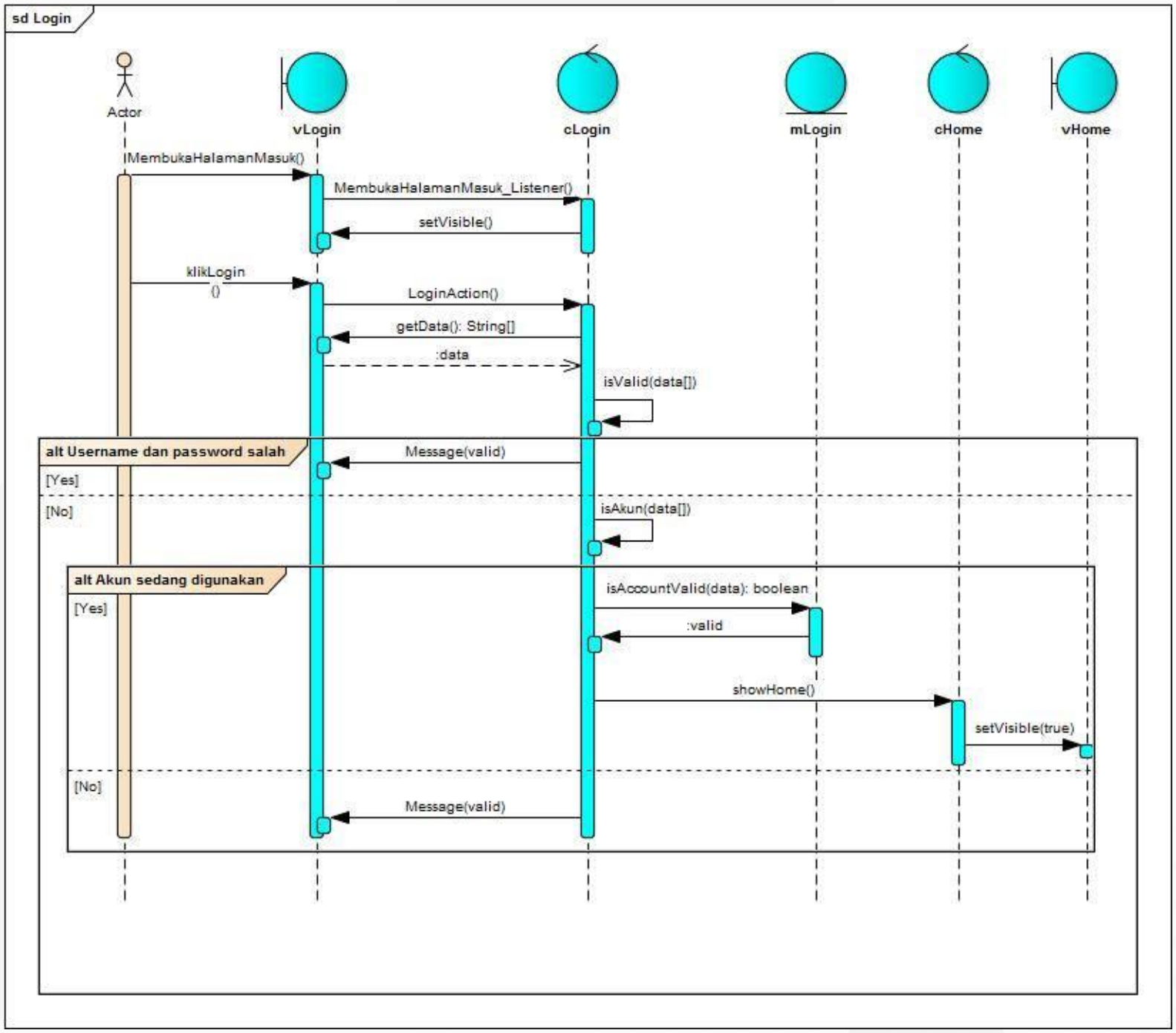
BEHAVIORAL MODEL (USE CASE DIAGRAM)



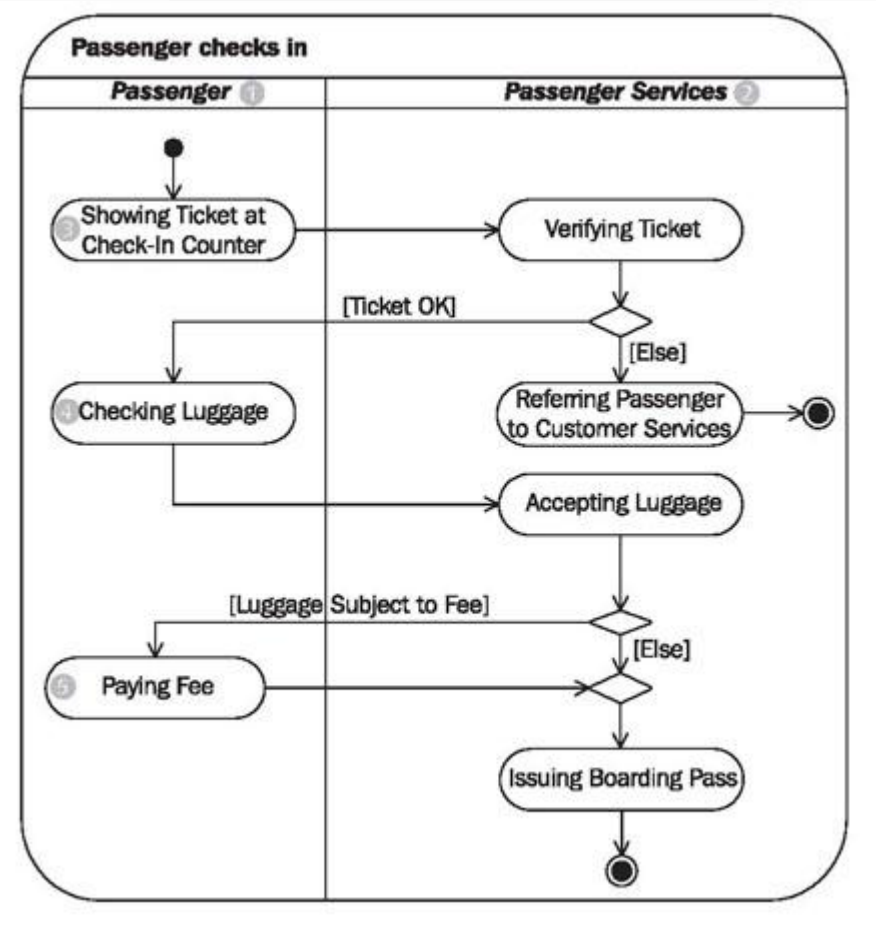
BEHAVIORAL MODEL (SEQUENCE DIAGRAM)

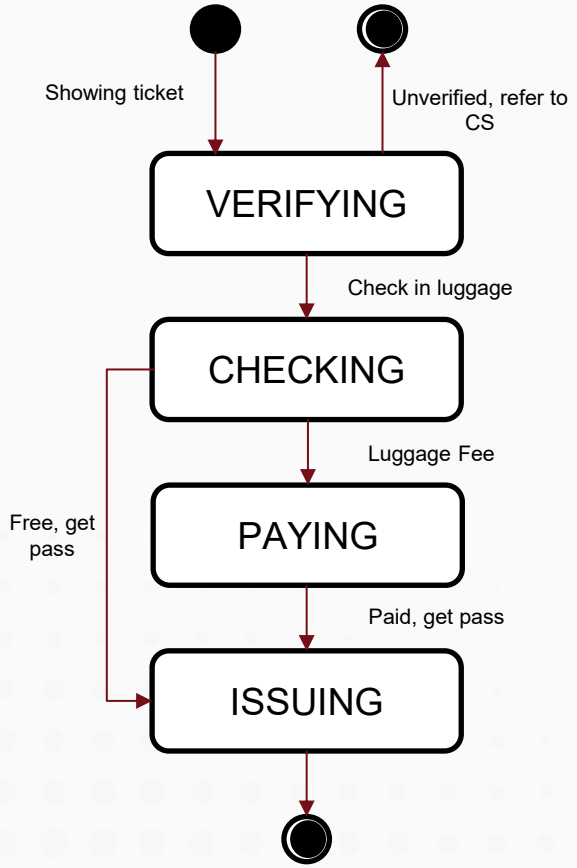


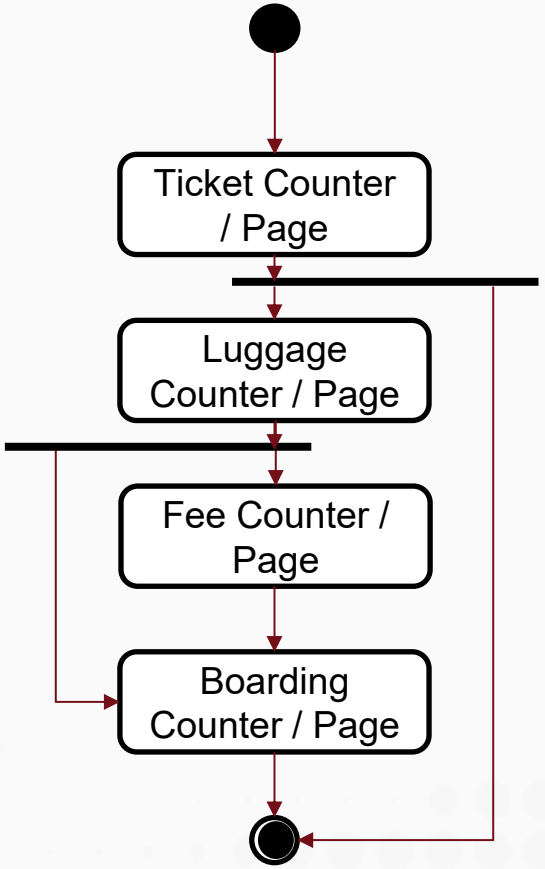
BEHAVIORAL MODEL (SEQUENCE DIAGRAM)



BEHAVIORAL MODEL (ACTIVITY vs STATE)







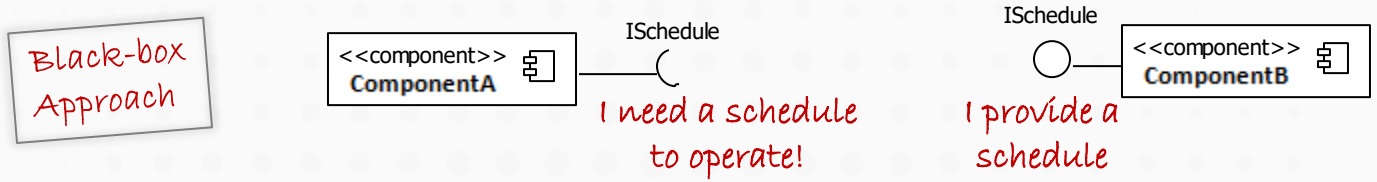
STRUCTURE MODEL (COMPONENT DIAGRAM)

- A component represents a *modular* part of a system that *encapsulates its contents* and whose manifestation is *replaceable* within its environment.

- Component diagrams are used to model software as group of components connected to each other through well-defined interfaces. They help decompose systems and represent their structural architecture from a logical perspective.

- Components can be modeled using an external black-box or internal white-box approach.
  - ✓ Black-box approach hides the component's internal structure.
    - Components interact with each other only through identified interfaces.
  - ✓ White-box approach shows the component's internal structure (e.g., realizing classifiers).

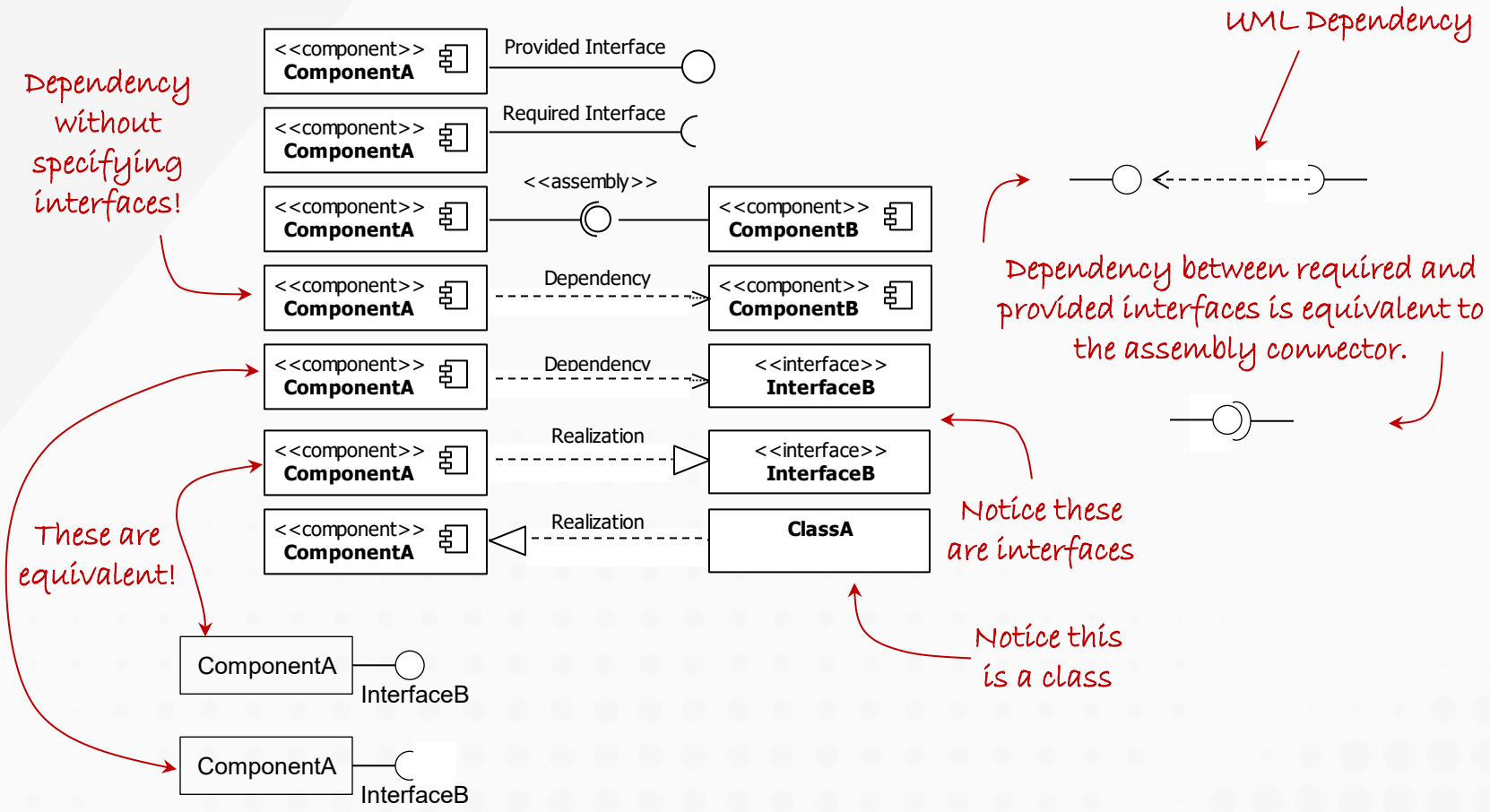
- Component interfaces are classified as *provided* or *required* interfaces.
  - ✓ Required interfaces are those the components need to carry out their functions.
  - ✓ Provided interfaces are used by other external components to interact with the component providing the services.



- Let's formally see how the UML relationships defined in previous sessions apply to the component classifier…

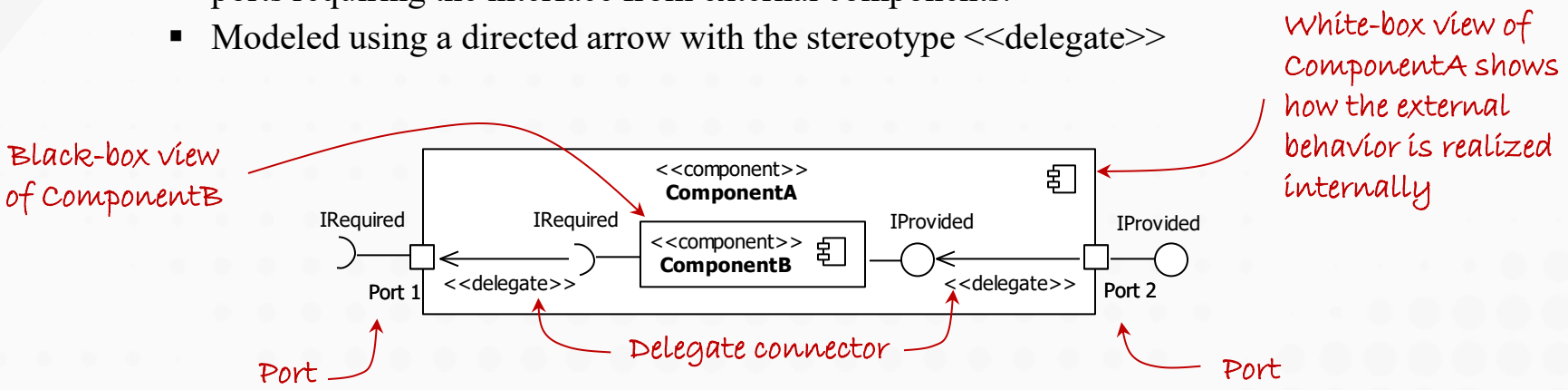
STRUCTURE MODEL (COMPONENT DIAGRAM)

UML relationships applied to the component classifier.

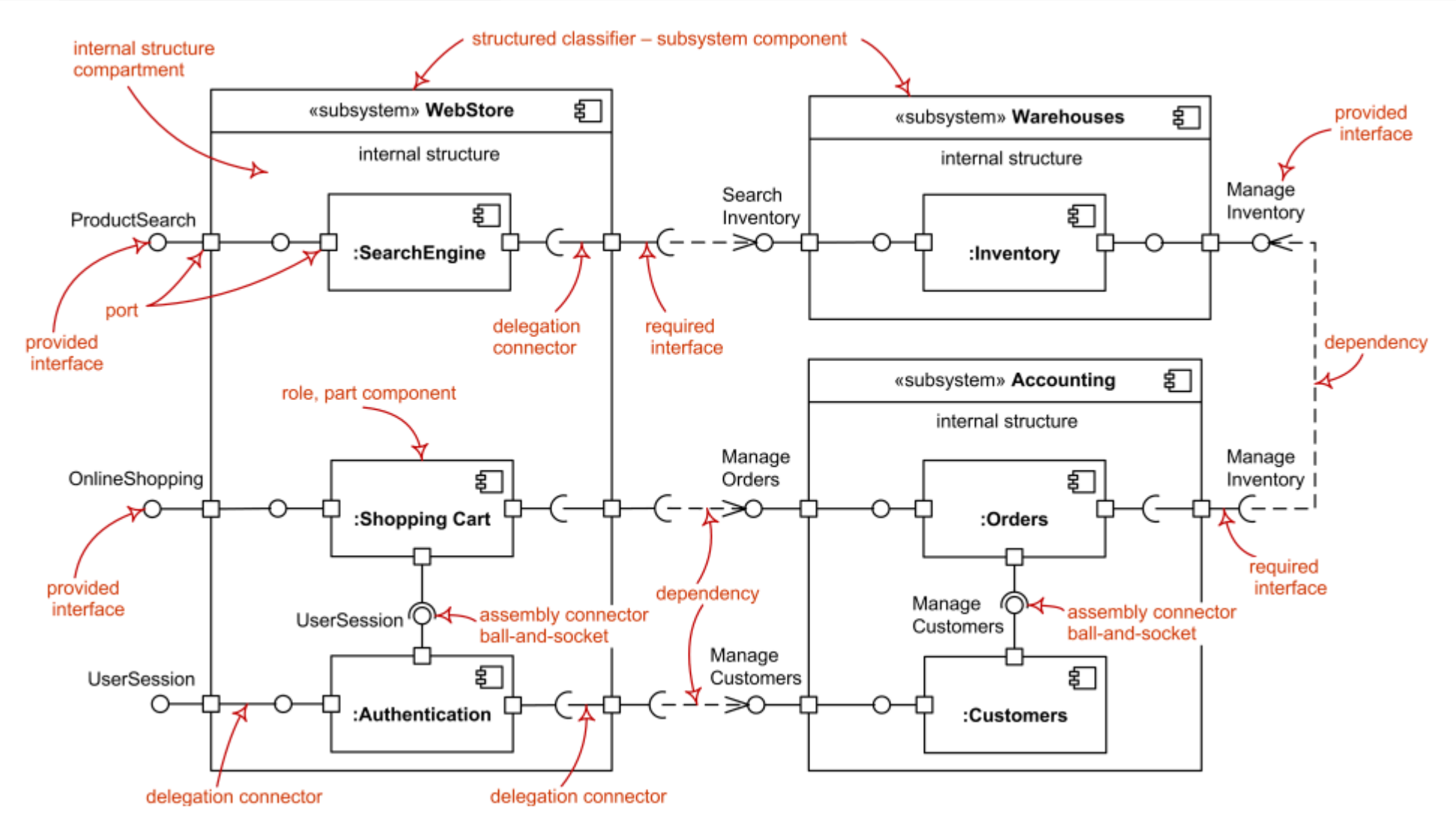


STRUCTURE MODEL (COMPONENT DIAGRAM)

- Two more important concepts used in component diagrams are:
  - ✓ Ports
    - Abstraction used to model access points for allowing the external environment to access the component's services and for allowing components to interact with their external environment.
    - Modeled using a small square at the boundary of a *classifier*, in this case, a component.
    - Ports can be named, e.g., port names below are Port 1 and Port 2.
  - ✓ Delegation connectors
    - Used to model the link between the external provided interfaces of a component to the realization of those interfaces internally within the component.
    - Similarly, delegation connectors model the link between internally required interfaces to ports requiring the interface from external components.
    - Modeled using a directed arrow with the stereotype <<delegate>>



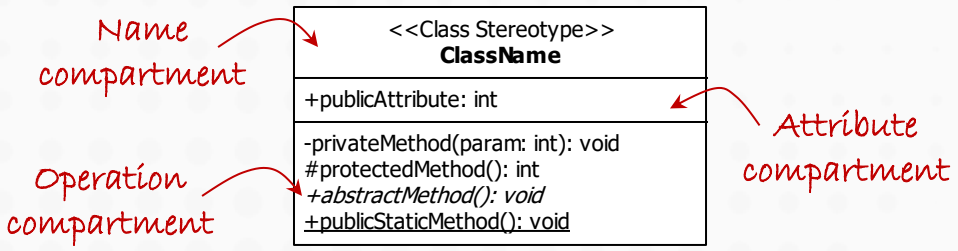
STRUCTURE MODEL (COMPONENT DIAGRAM)



STRUCTURE MODEL (CLASS DIAGRAM)

- Class diagrams exist at a lower level of abstraction than component diagrams.
  - ✓ Models consisting of classes and relationships between them necessary to achieve a system's functionality.
  - ✓ Weather a class diagram is created or not, the code of an object-oriented systems will always reflect some class design.
  - ✓ Therefore, there is two-way relationship between class diagrams and object-oriented code.
    - Class diagrams can be transformed to code (i.e., forward engineering)
    - Code can be transformed into class diagrams (i.e., reverse engineering)
  - ✓ This makes class diagrams the most powerful tool for designers to model the characteristics of object-oriented software before the construction phase.

- To become an effective designer, it is essential to understand the direct mapping between class diagrams and code. Let's take a closer look at the fundamental unit of the UML class diagram: the class.



STRUCTURE MODEL (CLASS DIAGRAM)

Example of the *forward engineering* of a UML class to C++ and Java.

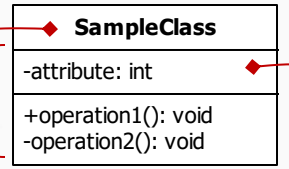
```
//  Generated by StarUML(tm) C++ Add-In
//
//  @ Project : Code Generation Tutorial
//  @ File Name : SampleClass.h
//  @ Date : 9/1/2012
//  @ Author : Carlos E. Otero
//
//


#if !defined(_SAMPLECLASS_H)
#define _SAMPLECLASS_H


class SampleClass {
public:
    void operation1();
private:
    int attribute;
    void operation2();
};

#endif  //_SAMPLECLASS_H
```

Code generated by free open source Star UML tool.



Class name

Private and public attributes and operations

Attribute name, type, and visibility

```
//  Generated by StarUML(tm) Java Add-In
//
//  @ Project : Code Generation Tutorial
//  @ File Name : SampleClass.java
//  @ Date : 9/1/2012
//  @ Author : Carlos E. Otero
//
//


public class SampleClass {
    private int attribute;
    public void operation1() {

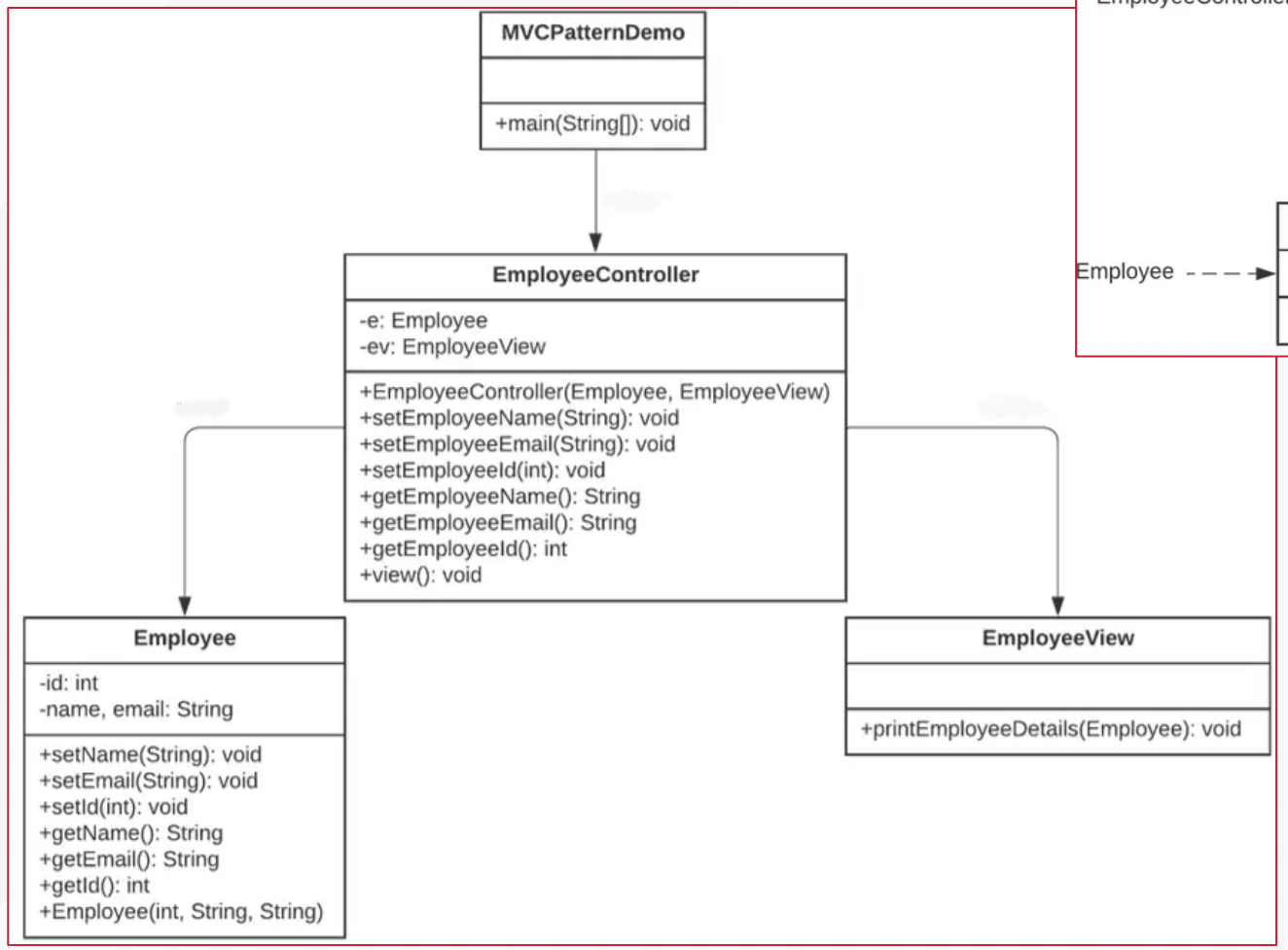
    }

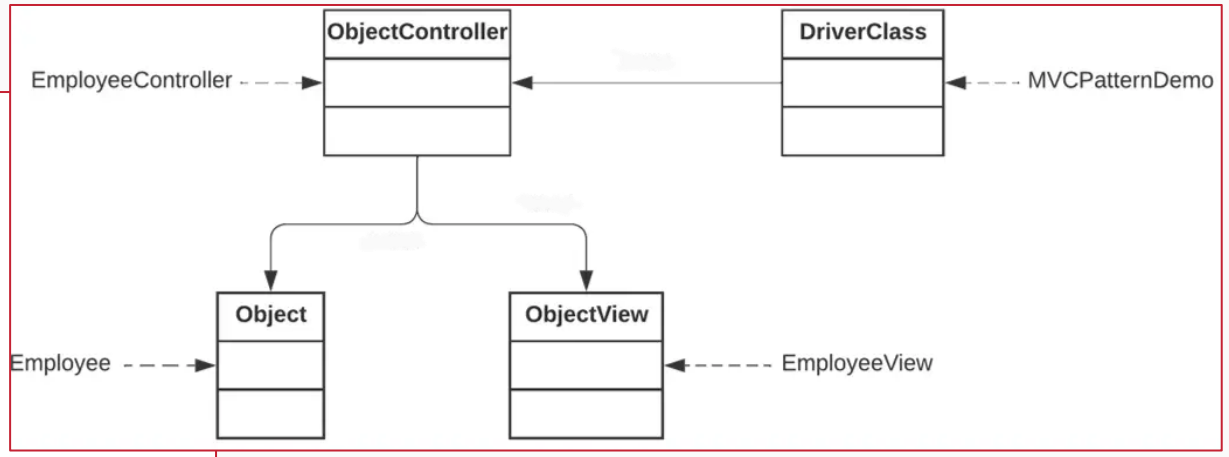
    private void operation2() {

    }
}
```

Important:
Notice how the modeled visibility {-, +} next to attribute and operations translate to code!

STRUCTURE MODEL (CLASS DIAGRAM)







Thank you for your participation. I am very grateful to have the opportunity into this sharing session. If there any enquiries, feels free to contact me.

+013 509 9215

khatibsyarbini@utm.my

THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

utm.my univteknologimalaysia utmofficial